

SÍNTESIS – LENGUAJES DE MARCAS - CSS

1. Integración y Anatomía Básica

Formas de aplicar CSS:

- **Externa:** Archivo .css independiente (etiqueta <link>). Es la más recomendada.
- **Interna:** Dentro del <head> (etiqueta <style>).
- **En línea:** Directamente en la etiqueta HTML (atributo style).
- **Por defecto:** Estilos base del propio navegador.

Anatomía de la regla CSS: Selector { Propiedad: Valor; }

2. Resolución de Conflictos (La Cascada)

Cuando varias reglas chocan, el navegador decide cuál aplicar según este orden de prioridad (de más a menos fuerte):

1. **!important:** Obliga a que esa regla gane por encima de todo.
2. **Especificidad (Selectores):** Un ID (#) gana a una Clase (.), y una Clase gana a una Etiqueta (p).
3. **Localidad:** El estilo en línea gana al interno, y el interno al externo.
4. **Orden de lectura:** A igualdad de condiciones, gana la última regla escrita (la que está más abajo en el código).

3. Estructura y Modelo de Caja

Todo en CSS es una caja.

Tipos de visualización (display):

- **Bloque (block):** Ocupan todo el ancho disponible, añaden salto de línea antes y después. Elemento genérico: <div>.
- **En línea (inline):** Ocupan solo lo que ocupa su contenido, no generan saltos de línea. Elemento genérico: .
- **Lista (list-item):** Como el bloque, pero con viñeta.

4. Selectores (Apuntando a los elementos)

Tipo de Selector	Sintaxis	¿A quién afecta?
Etiqueta	p, h1	A todos los elementos de ese tipo.
Clase	.nombreClase	Elementos con class="nombreClase". Reutilizable.
ID	#nombreID	Elemento con id="nombreID". Único en la página.
Hijos (CSS2)	padre > hijo	Descendiente directo.
Hermanos (CSS2)	caja1 + caja2	Elemento inmediatamente posterior al mismo nivel.
Atributos (CSS2)	[type="text"]	Elementos que posean ese atributo/valor.

Pseudo-clases (Estados virtuales): Dependen de la interacción (:hover, :active, :link, :visited, :focus).

Pseudo-elementos (Partes visuales): Afectan a fragmentos (:first-letter, :first-line). *Deben ir al final del selector.*

5. Posicionamiento (Ubicando las cajas)

El atributo **position** dicta cómo se colocan las cajas en la pantalla.

Posición	Comportamiento en el flujo	Referencia para moverse (top, left...)
Static (Por defecto)	Dentro del flujo (orden normal).	Ninguna (ignora top, left, etc.).
Relative	Dentro del flujo (deja su hueco fantasma).	Su propia posición original.
Absolute	Fuera del flujo (no deja hueco).	Su contenedor padre más cercano posicionado (que no sea static).
Fixed	Fuera del flujo (no deja hueco).	La ventana del navegador (viewport). Resiste el scroll.

Flotabilidad (float):

- Saca el elemento del flujo para que el texto lo rodee (left, right, none).
- **Control de limpieza (clear):** Obliga a un elemento a colocarse debajo de los elementos flotantes (left, right, both). *Nota: Prácticamente obsoleto para layouts estructurales, reemplazado por Flexbox/Grid.*

6. Unidades y Medios (CSS2)

Unidades de Medida:

- **Relativas:** Flexibles y recomendadas (em para fuente, ex, px píxeles, % porcentajes).
- **Absolutas:** Rígidas, útiles solo si conoces el medio físico (cm, mm, pt, in).

Dispositivos (Media Queries):

Permiten adaptar el diseño al medio mediante el atributo media en el HTML o la regla @media en el CSS.

- *Valores comunes:* screen (pantallas), print (impresoras), handheld (móviles antiguos/pantallas pequeñas).

7. Variables CSS (Custom Properties)

Funcionan como un "almacén" de valores que te ahorra repetir código y facilita los cambios globales (ej. implementar un modo oscuro).

Sintaxis y Uso:

- **Declaración:** Se definen en el elemento raíz :root (para que sean globales) y siempre empiezan con dos guiones --.
 - --color-primario: #1D3557;
- **Aplicación:** Se llaman utilizando la función var().
 - color: **var**(--color-primario);

Buenas Prácticas:

- Organizar por categorías (ej. tipografía, colores, espacios).
- Comentarlas para facilitar el trabajo en equipo.

8. Profundizando en position: fixed

- **Comportamiento exacto:** Se ancla a la ventana del navegador (**viewport**), ignorando el flujo del documento.
- **Usos ideales:** Menús pegajosos, botones de "volver arriba", modales de cookies.
- **Peligros y precauciones:**

- Como no ocupa espacio, **debes dar márgenes al contenido principal para que el elemento fixed no lo tape.**
- **Requiere dominar el z-index** (el eje Z de profundidad) para asegurar que quede por encima de todo.
- **En móviles:** Suele estorbar mucho. La alternativa moderna y más amigable suele ser `position: sticky`; (se comporta como `relative` hasta que haces scroll y toca un límite, entonces se vuelve `fixed`).

9. El Declive de float y la Maquetación Moderna

Al estudiar las técnicas de maquetación web actuales, es vital entender la evolución histórica para no caer en malas prácticas.

- **El propósito original:** Hacer que el texto rodeara una imagen (como en un periódico impreso).
- **El "Hack" de los 2000s:** Se usó para forzar estructuras de columnas. Esto provocaba que los contenedores colapsaran (perdieran su altura) y obligaba a usar parches engorrosos de código (el famoso truco *Clearfix*).
- **El Estándar Actual:** float vuelve a ser solo para imágenes. Para estructurar la web hoy en día, las tecnologías clave son:
 - **display: flex:** Para alinear elementos en **una dimensión** (una sola fila o una sola columna).
 - **display: grid:** Para estructuras complejas en **dos dimensiones** (filas y columnas al mismo tiempo).

10. Media Queries y Diseño Responsive

Son las reglas condicionales que hacen que tu web sea "elástica" y se adapte al dispositivo desde el que se está viendo.

Los dos grandes enfoques:

Enfoque	Técnica CSS	Filosofía
Mobile First <i>(Recomendado)</i>	min-width	Diseñas la versión móvil por defecto (fuera de la media query). Luego, añades complejidad visual cuando la pantalla crece a tablet o escritorio. Mejora la carga y prioriza lo esencial.
Desktop First	max-width	Diseñas el escritorio por defecto y vas quitando/comprimiendo elementos cuando la pantalla se hace pequeña.

Buenas Prácticas para Media Queries:

1. **Ubicación:** Ponlas siempre al final de tu hoja de estilos CSS para que no sean sobrescritas accidentalmente por la cascada.
2. **Propósito:** Úsalas para alterar la estructura (ej. pasar de apilamiento vertical en móvil a 3 columnas horizontales en PC), no solo para cambiar el color de una letra.
3. **Filosofía fluida:** Tu diseño base ya debería ser elástico usando unidades relativas (% , rem, em, vh, vw). Las media queries solo deben usarse en los "puntos de rotura" (*breakpoints*) donde el diseño fluido ya no se sostenga y se rompa.

11. Flexbox vs. Grid: La Gran Diferencia

- **Flexbox (1 Dimensión):** Trabaja en una sola dirección a la vez (o una fila, o una columna). Es perfecto para alinear elementos dentro de componentes pequeños (menús, tarjetas, barras de navegación).
- **Grid (2 Dimensiones):** Trabaja con filas y columnas simultáneamente. Es el arquitecto perfecto para estructurar el *layout* general de la página entera.

12. CSS Flexbox: El Rey de la Alineación

Todo empieza con un **Eje Principal** (hacia dónde fluyen las cosas) y un **Eje Secundario** (el perpendicular).

A. Propiedades del PADRE (display: flex;)

Propiedad	¿Qué hace?	Valores Clave
flex-direction	Define el Eje Principal.	row (→), column (↓), row-reverse (←), column-reverse (↑).
flex-wrap	Comportamiento si no caben.	nowrap (se aprietan), wrap (bajan a otra línea).
flex-flow	<i>Atajo de las dos anteriores.</i>	Ej: flex-flow: row wrap;

Alineación en el contenedor:

Propiedad	¿Dónde alinea?	Comportamientos (Valores)
justify-content	En el Eje Principal	flex-start (inicio), flex-end (final), center (centro), space-between (separados al límite), space-around (espacio igual alrededor).
align-items	En el Eje Secundario	flex-start, flex-end, center, stretch (se estiran para rellenar, <i>por defecto</i>).
align-content	Bloques Multilínea	Igual que <i>justify-content</i> , pero actúa sobre las filas enteras cuando hay wrap.

B. Propiedades de los HIJOS (Ítems)

- **align-self:** El "rebelde". Sobrescribe el align-items del padre solo para este ítem en concreto.

- **order:** Cambia su posición visual numéricamente (ej. order: 1) sin tener que tocar el código HTML.
- **Gestión del Tamaño (El atajo flex):**
 - **flex-grow:** Cuánto **crece** si sobra espacio.
 - **flex-shrink:** Cuánto **encoge** si falta espacio.
 - **flex-basis:** Su tamaño ideal de partida.
 - **Sintaxis del atajo:** flex: grow shrink basis; (Ej: flex: 1 0 50%;)

13. CSS Grid: El Arquitecto de Cuadrículas

La cuadrícula se compone de **Celdas** (un cuadrito), **Líneas** (los separadores invisibles), **Bandas** (filas/columnas enteras) y **Áreas** (agrupaciones de varias celdas).

A. Propiedades del PADRE (display: grid;)

- **Estructura Base:**
 - **grid-template-columns:** Define el ancho de las columnas.
 - **grid-template-rows:** Define el alto de las filas.
 - *Truco:* Usa **fracciones (fr)** y la **función repeat()**. Ej: repeat(3, 1fr) crea 3 columnas exactamente iguales.
- **Huecos (gap):**
 - Sustituye a los engorrosos márgenes. Separa las celdas limpiamente con grid-column-gap, grid-row-gap, o simplemente su atajo **gap**.
- **Plantillas Visuales (grid-template-areas):**
 - Te permite "dibujar" el diseño usando nombres de texto. (Ej. poner "header header" arriba y "menu contenido" abajo).

B. Propiedades de los HIJOS (Ítems)

Para decirle a un hijo dónde debe colocarse dentro de la cuadrícula del padre, tienes dos métodos:

Método 1: Por Líneas (Numérico)

Le dices en qué línea divisoria empieza y en cuál acaba.

- **grid-column:** 1 / 3; (Empieza en la línea 1 vertical, acaba en la 3).
- **grid-row:** 2 / 4; (Empieza en la línea 2 horizontal, acaba en la 4).
- **Palabra clave span:** Sirve para "expandirse". Ej: grid-column: span 2; (Ocupa 2 columnas desde donde estés).

Método 2: Por Áreas (Nominal)

- **grid-area:** nombreAsignado; (Le das un nombre al hijo para que el padre lo ubique automáticamente en su "dibujo" de grid-template-areas).

(Al igual que en Flexbox, los hijos en Grid también pueden usar justify-self y align-self para moverse de forma independiente dentro de su propia celda).

14. El Rol del Movimiento (UX/UI)

La animación no es decoración, es comunicación. Si no cumple una de estas 4 reglas, probablemente sobre:

1. **Guiar:** Llevar el ojo al botón importante.
2. **Feedback:** Confirmar que el clic funcionó (el botón reacciona).
3. **Carga Cognitiva:** Suavizar cambios bruscos para no marear al usuario.
4. **Estado del Sistema:** Avisar de que algo está cargando (loaders) para evitar la frustración.

15. Animaciones Nativas con CSS (El Motor Básico)

Tienes dos herramientas integradas en CSS. Piensa en ellas como un "Interruptor" frente a una "Película".

Característica	Transiciones (transition)	Keyframes (@keyframes + animation)
¿Cómo funciona?	Interpola de un estado A a un estado B.	Permite múltiples paradas (0%, 50%, 100%).
¿Cuándo se activa?	Requiere un detonante (ej. hacer :hover).	Puede ejecutarse solo, al cargar la página o en bucle.
Complejidad	Baja (Microinteracciones directas).	Alta (Movimientos autónomos o complejos).
Atajo habitual	transition: all 0.3s ease;	animation: latido 2s infinite alternate;

Nota sobre animation-fill-mode: forwards: Es vital para que, cuando la animación termine, el elemento se quede congelado en el final y no regrese de golpe al principio.

16. Librerías de Animación (El Arsenal)

Cuando el CSS nativo se queda corto o necesitas ir muy rápido, usas librerías. Aquí comparamos las dos más famosas:

A. Animate.css (La solución rápida - "Plug & Play")

- **Qué es:** Un archivo CSS lleno de @keyframes ya creados por otros.
- **Cómo se usa:** Añadiendo clases HTML. Siempre animate__animated + la clase del efecto (animate__fadeIn, animate__bounce).
- **Ajustes:** Tiene clases extra para controlar el retardo (animate__delay-1s), velocidad (animate__fast) o repetición (animate__infinite).
- **Ideal para:** Prototipos, landings, mensajes de error, animar entradas al cargar la web.

B. GSAP - GreenSock (El estándar profesional)

Librería de JavaScript, permite sincronizar animaciones complejas y reaccionar al usuario (ej. animar al hacer scroll).

- **Los 4 Comandos Mágicos:**

- **gsap.set()**: Teletransporta el elemento a una posición inicial al instante.
- **gsap.to()**: Anima el elemento **hacia** los valores que le digas.
- **gsap.from()**: El elemento viene **desde** los valores que le digas hacia su estado normal en CSS. (Muy útil para entradas).
- **gsap.fromTo()**: Control absoluto (A $\rightarrow$ B ignorando el CSS).

- **Conceptos:**

- **stagger**: Si animas una lista, hace que entren en cascada (uno detrás de otro con una ligera pausa), no todos de golpe.
- **Timelines (gsap.timeline())**: Es el "Coreógrafo". En lugar de calcular retardos manualmente, encadena animaciones .to().to().to() y GSAP hace que una empiece justo cuando acaba la anterior.
- **Callbacks**: Funciones que saltan en momentos clave (onStart, onComplete).

17. Patrones de Diseño (Trucos Modernos)

Son "recetas" que resuelven problemas comunes en la maquetación actual:

- **El Centrado Perfecto**: display: grid; place-items: center;. Olvídate de los viejos trucos; esto centra cualquier cosa tanto horizontal como verticalmente en dos líneas.
- **Cajas Predecibles**: box-sizing: border-box;. Obligatorio. Hace que el padding y el borde se coman espacio hacia adentro, evitando que tus cajas crezcan y rompan el diseño.

- **Variables (El Mando a Distancia):** Definir colores o medidas en :root con --nombre. Si cambias ese valor, cambia en toda la web a la vez.
- **Efecto Cristal (Glassmorphism):** Un fondo semitransparente + backdrop-filter: blur(10px);.
- **Capas Fantasma:** pointer-events: none;. Hace que el elemento sea visual, pero "traspasable" para el ratón (puedes hacer clic en lo que hay debajo).

18. JavaScript y el Rendimiento (El Santo Grial de los 60 FPS)

Animar mal en la web hace que el ordenador del usuario se sobrecaliente o que la animación vaya a saltos (el temido *jank*). Para mantener la fluidez de 60 Fotogramas Por Segundo (FPS), sigue esta regla de oro:

La Regla de la GPU (Aceleración Gráfica):

-  **LO QUE DEBES ANIMAR: transform** (para mover, escalar, rotar) y **opacity** (para desvanecer). Estas propiedades las procesa la Tarjeta Gráfica (GPU), que es rapidísima.
-  **LO QUE NUNCA DEBES ANIMAR: width, height, margin, top, left.** Animar esto obliga a la CPU a recalcular toda la estructura de la página (*Layout*) 60 veces por segundo, arruinando el rendimiento.

Optimizaciones Extra:

- **Selectores:** getElementById() es el más rápido. querySelector() es más cómodo y versátil.
- **Throttling:** Si ejecutas código con el movimiento del ratón, debes ponerle un "freno" (ej. usar performance.now() para que solo se ejecute cada 30ms) o saturarás la CPU.
- **Pre-aviso:** will-change: transform; le dice al navegador: "Oye, prepara recursos porque este elemento se va a mover pronto". (*Úsalo con pinzas, solo en elementos críticos*).